Decorator

Decorator · Stack compression + encryption on a data store · Build with Waqar

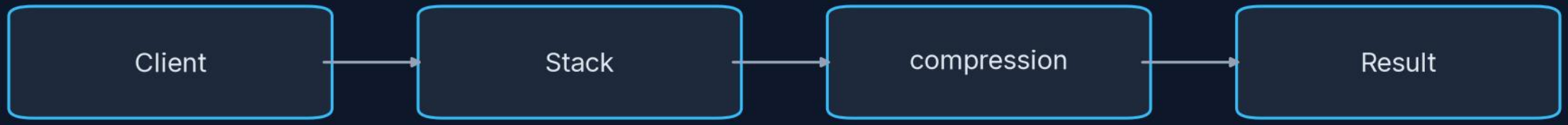
Today's Hook

- Problem: Stack compression + encryption on a data store
- Pattern: Decorator — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Stack compression + encryption on a data store
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Decorator: Stack compression + encryption on a data store

Real-World Scenario

- Running example: Stack compression + encryption on a data store
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.structural.decorator;
import com.buildwithwaqar.patterns.gof.structural.decorator.*;
/** Problem: store secrets with compression then encryption without changing client code. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Decorator problem: compress+encrypt store ===");
    DataSource ds = DataSourceDecorators.encryptedCompressedMemory();
    ds.write("token = super secret");
    System.out.println("roundTrip=" + ds.read());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.structural.decorator;

public interface DataSource {
    String read();
    void write(String data);
}


package com.buildwithwaqar.patterns.gof.structural.decorator;

final class InMemoryDataSource implements DataSource {
    private String data = "";
    public String read() { return data; }
    public void write(String data) { this.data = data; }
}

abstract class DataSourceDecorator implements DataSource {
    protected final DataSource wrappee;
    protected DataSourceDecorator(DataSource wrappee) { this.wrappee = wrappee; }
    public String read() { return wrappee.read(); }
    public void write(String data) { wrappee.write(data); }
}
```

Solution (2)

```
final class EncryptionDecorator extends DataSourceDecorator {
    EncryptionDecorator(DataSource wrappee) { super(wrappee); }

    @Override public void write(String data) {
        super.write(new StringBuilder(data).reverse().toString());
    }

    @Override public String read() {
        return new StringBuilder(super.read()).reverse().toString();
    }
}

/** Stackable decorator demo entry — compression over encryption over memory. */
public final class DataSourceDecorators {
    private DataSourceDecorators() {}

    public static DataSource encryptedCompressedMemory() {
        return new EncryptionDecorator(new CompressionDecorator(new InMemoryDataSource()));
    }
}
```

Solution (3)

```
final class CompressionDecorator extends DataSourceDecorator {
    CompressionDecorator(DataSource wrappee) { super(wrappee); }

    @Override public void write(String data) {
        super.write("c:" + data.replace(" ", ""));
    }

    @Override public String read() {
        var s = super.read();
        return s.startsWith("c:") ? s.substring(2) : s;
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.structural.decorator;
import com.buildwithwaqar.patterns.gof.structural.decorator.*;
/** Problem: store secrets with compression then encryption without changing client code. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Decorator problem: compress+encrypt store ===");
    DataSource ds = DataSourceDecorators.encryptedCompressedMemory();
    ds.write("token = super secret");
    System.out.println("roundTrip=" + ds.read());
  }
}
```

Walkthrough Output

```
=== Decorator problem: compress+encrypt store ===
roundTrip=token=supersecret
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Decorator.

Related Patterns

- Contrast Decorator with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Decorator → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.